

Clasificación del Consumo de Psicofármacos con Machine Learning

Objetivo

El objetivo de este trabajo es aplicar modelos de aprendizaje automático supervisado para clasificar el consumo de psicofármacos en Argentina, comparando el desempeño de dos algoritmos:

- K-Nearest Neighbors (KNN)
- Árbol de Decisión

Se analizarán métricas de evaluación y matrices de confusión para determinar cuál modelo obtiene mejores resultados.

Dataset utilizado

Dataset: psicofarmacos_argentina.csv

El dataset contiene información relacionada con:

- Ansiolíticos
- Antidepresivos
- Antipsicóticos
- Hipnóticos
- Consumo total

Cantidad de registros:

- 18 filas

Cantidad de variables:

- 7 columnas

Variable objetivo

La variable original `total_consumo` fue transformada en una clasificación binaria:

- Consumo bajo → clase 0
- Consumo alto → clase 1

Esto permite trabajar con algoritmos de clasificación supervisada.

IMPORTACION DE LIBRERIAS

```
In [39]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)
```

CARGA DEL SET DE DATOS

```
In [2]: df = pd.read_csv("psicofarmacos_argentina.csv")

df.head()
```

```
Out[2]:
```

	anio	mes	ansioliticos	antidepresivos	antipsicoticos	hipnoticos	total_consumo
0	2023	1	120	95	40	60	315
1	2023	2	125	98	42	62	327
2	2023	3	130	102	43	65	340
3	2023	4	128	100	45	63	336
4	2023	5	135	105	46	68	354

CREACION DE LA VARIABLE BINARIA

```
In [5]: df.columns
```

```
Out[5]: Index(['anio', 'mes', 'ansioliticos', 'antidepresivos', 'antipsicoticos',
               'hipnoticos', 'total_consumo', 'consumo_alto'],
              dtype='object')
```

```
In [6]: df.columns = df.columns.str.strip()
```

```
df.columns
```

```
Out[6]: Index(['anio', 'mes', 'ansioliticos', 'antidepresivos', 'antipsicoticos',  
             'hipnoticos', 'total_consumo', 'consumo_alto'],  
            dtype='object')
```

```
In [7]: mediana = df["total_consumo"].median()
```

```
df["consumo_alto"] = np.where(  
    df["total_consumo"] >= mediana,  
    1,  
    0  
)
```

```
df[[  
    "total_consumo",  
    "consumo_alto"  
]]
```

```
Out[7]:
```

	total_consumo	consumo_alto
--	---------------	--------------

0	315	0
1	327	0
2	340	0
3	336	0
4	354	0
5	368	0
6	382	0
7	397	0
8	391	0
9	411	1
10	425	1
11	439	1
12	455	1
13	470	1
14	485	1
15	479	1
16	499	1
17	515	1

MOTRAR TABLA

```
In [10]: df[[  
    "total_consumo",
```

```
"consumo_alto"  
]].head()
```

Out[10]:

	total_consumo	consumo_alto
0	315	0
1	327	0
2	340	0
3	336	0
4	354	0

DISTRIBUCION DE CLASES

In [13]: `df["consumo_alto"].value_counts().to_frame()`

Out[13]:

	count
consumo_alto	
0	9
1	9

Selección de variables

Variables predictoras:

- * ansioliticos
- * antidepresivos
- * antipsicoticos
- * hipnoticos

Variable objetivo:

consumo_alto

In [14]:

```
X = df[[  
    "ansioliticos",  
    "antidepresivos",  
    "antipsicoticos",  
    "hipnoticos"  
]]  
  
y = df["consumo_alto"]  
  
X.head()
```

Out[14]:

	ansioliticos	antidepresivos	antipsicoticos	hipnoticos
0	120	95	40	60
1	125	98	42	62
2	130	102	43	65
3	128	100	45	63
4	135	105	46	68

División de entrenamiento y prueba

Se utilizó una división:

80% entrenamiento

20% prueba

```
In [15]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42
    )

    pd.DataFrame({
        "Conjunto": ["Entrenamiento", "Prueba"],
        "Cantidad": [len(X_train), len(X_test)]
    })
```

Out[15]:

	Conjunto	Cantidad
0	Entrenamiento	14
1	Prueba	4

Escalado de datos

El escalado es necesario para el modelo KNN.

```
In [17]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

pd.DataFrame(
    X_train_scaled,
    columns=X.columns
).head()
```

```
Out[17]:
```

	ansioliticos	antidepresivos	antipsicoticos	hipnoticos
0	-1.636783	-1.585949	-1.367796	-1.718147
1	0.735471	0.737651	0.679597	0.787484
2	1.240205	1.254007	1.281771	1.188385
3	0.886891	0.892558	0.920466	0.887709
4	0.230736	0.221295	0.077422	0.186133

Modelo K-Nearest Neighbors (KNN)

Configuración:

`n_neighbors = 3`

`weights = distance`

```
In [20]: knn = KNeighborsClassifier(
          n_neighbors=3
        )

knn.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)

y_pred_knn
```

```
Out[20]: array([0, 0, 0, 0])
```

Evaluación del modelo KNN

```
In [24]: pd.DataFrame({
          "Métrica": ["Accuracy KNN"],
          "Valor": [
              accuracy_score(
                  y_test,
                  y_pred_knn
              )
          ]
        })
```

```
Out[24]:
```

	Métrica	Valor
0	Accuracy KNN	1.0

```
In [23]: reporte_knn = pd.DataFrame(
          classification_report(
              y_test,
              y_pred_knn,
              output_dict=True
          )
        ).transpose()
```

reporte_knn

Out[23]:

	precision	recall	f1-score	support
0	1.0	1.0	1.0	4.0
accuracy	1.0	1.0	1.0	1.0
macro avg	1.0	1.0	1.0	4.0
weighted avg	1.0	1.0	1.0	4.0

Matriz de confusión KNN

```
In [41]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
```

```
cm_knn = confusion_matrix(
    y_test,
    y_pred_knn,
    labels=[0,1]
)

plt.figure(figsize=(5,4))

plt.imshow(
    cm_knn,
    cmap="Reds"
)

plt.title(
    "Matriz de Confusión KNN",
    fontsize=14,
    color="#5C0000",
    fontweight="bold"
)

plt.colorbar()

classes = ["Bajo", "Alto"]

plt.xticks(
    np.arange(len(classes)),
    classes
)

plt.yticks(
    np.arange(len(classes)),
    classes
)
```

```

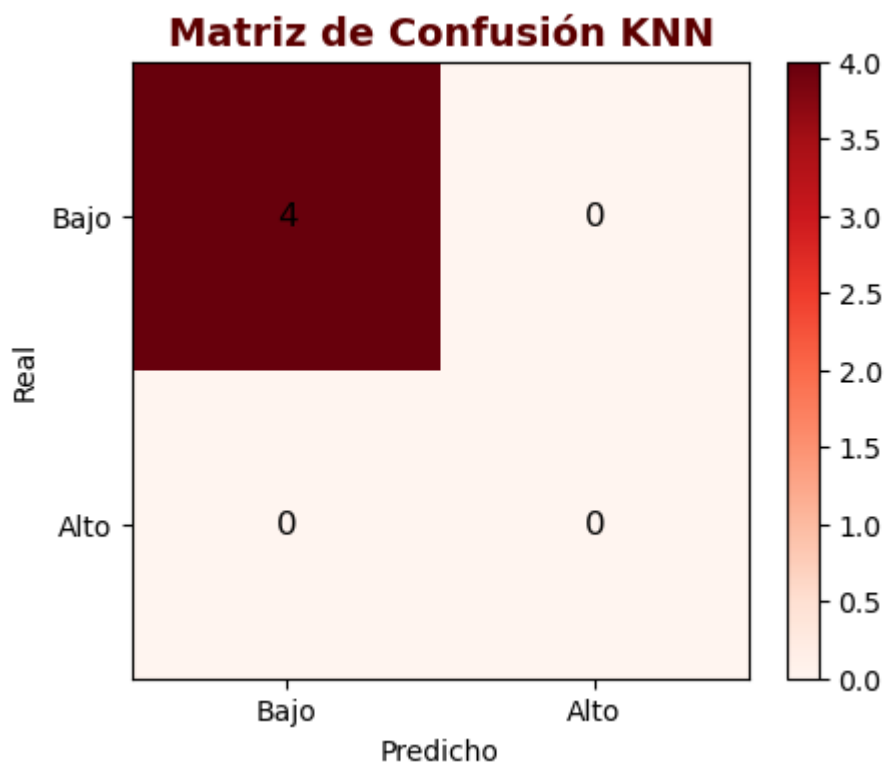
for i in range(cm_knn.shape[0]):
    for j in range(cm_knn.shape[1]):

        plt.text(
            j,
            i,
            cm_knn[i, j],
            ha="center",
            va="center",
            color="black",
            fontsize=12
        )

plt.xlabel("Predicho")
plt.ylabel("Real")

plt.show()

```



Modelo Árbol de Decisión

Configuración:

criterion = gini

max_depth = 3

```

In [34]: tree = DecisionTreeClassifier(
            criterion="gini",
            max_depth=3,
            random_state=42
        )

```



```
tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)

y_pred_tree
```

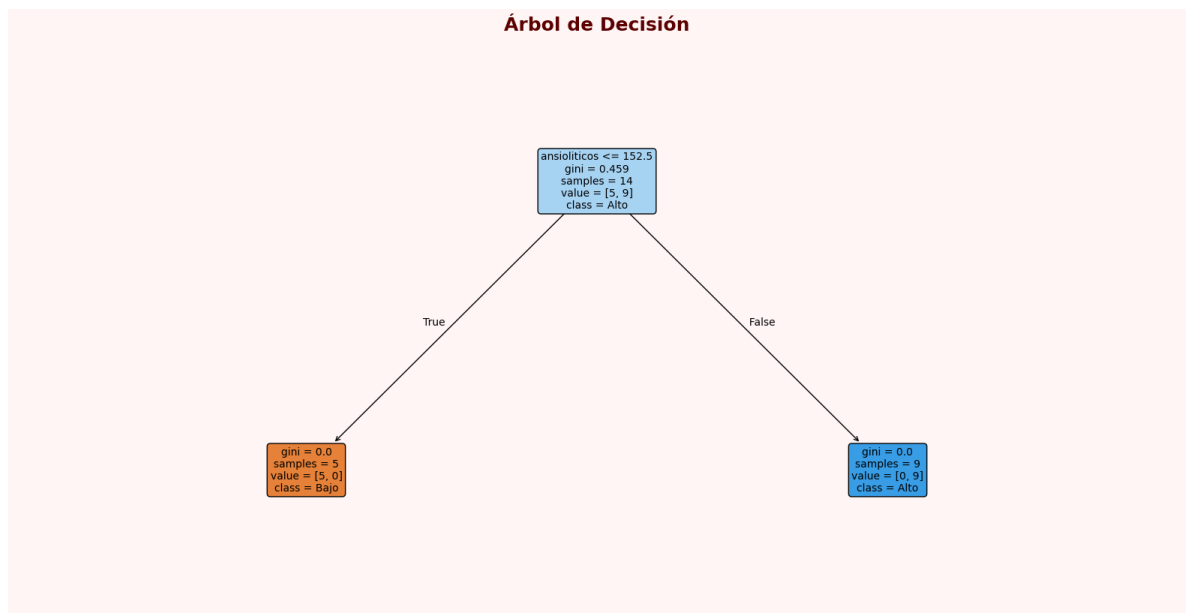
Out[34]: array([0, 0, 0, 0])

```
In [40]: plt.figure(
    figsize=(20,10),
    facecolor="#fff5f5"
)

plot_tree(
    tree,
    feature_names=X.columns,
    class_names=["Bajo", "Alto"],
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title(
    "Árbol de Decisión",
    fontsize=18,
    color="#5C0000",
    fontweight="bold"
)

plt.show()
```



Evaluación del Árbol de Decisión

```
In [35]: pd.DataFrame({
    "Métrica": ["Accuracy Árbol"],
    "Valor": [
        accuracy_score(
            y_test,
            y_pred_tree
        )
    ]
})
```

```
]
})
```

Out[35]:

	Métrica	Valor
0	Accuracy Árbol	1.0

```
In [38]: reporte_tree = pd.DataFrame(
    classification_report(
        y_test,
        y_pred_tree,
        output_dict=True
    )
).transpose()

reporte_tree
```

Out[38]:

	precision	recall	f1-score	support
0	1.0	1.0	1.0	4.0
accuracy	1.0	1.0	1.0	1.0
macro avg	1.0	1.0	1.0	4.0
weighted avg	1.0	1.0	1.0	4.0

Matriz de confusión Árbol de Decisión

```
In [43]: cm_tree = confusion_matrix(
    y_test,
    y_pred_tree
)

plt.figure(figsize=(5,4))

plt.imshow(
    cm_tree,
    cmap="OrRd"
)

plt.title(
    "Matriz de Confusión Árbol",
    color="#5C0000",
    fontsize=14,
    fontweight="bold"
)

plt.colorbar()

plt.xticks(
    [0,1],
    ["Bajo", "Alto"]
)

plt.yticks(
    [0,1],
    ["Bajo", "Alto"]
)
```

```

)

for i in range(2):
    for j in range(2):

        plt.text(
            j,
            i,
            cm_tree[i,j],
            ha="center",
            va="center",
            color="black"
        )

plt.xlabel("Predicho")
plt.ylabel("Real")

plt.show()

```

/opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-packages/sklearn/metrics/_classification.py:534: UserWarning: A single label was found in 'y_true' and 'y_pred'. For the confusion matrix to have the correct shape, use the 'labels' parameter to pass all known labels.

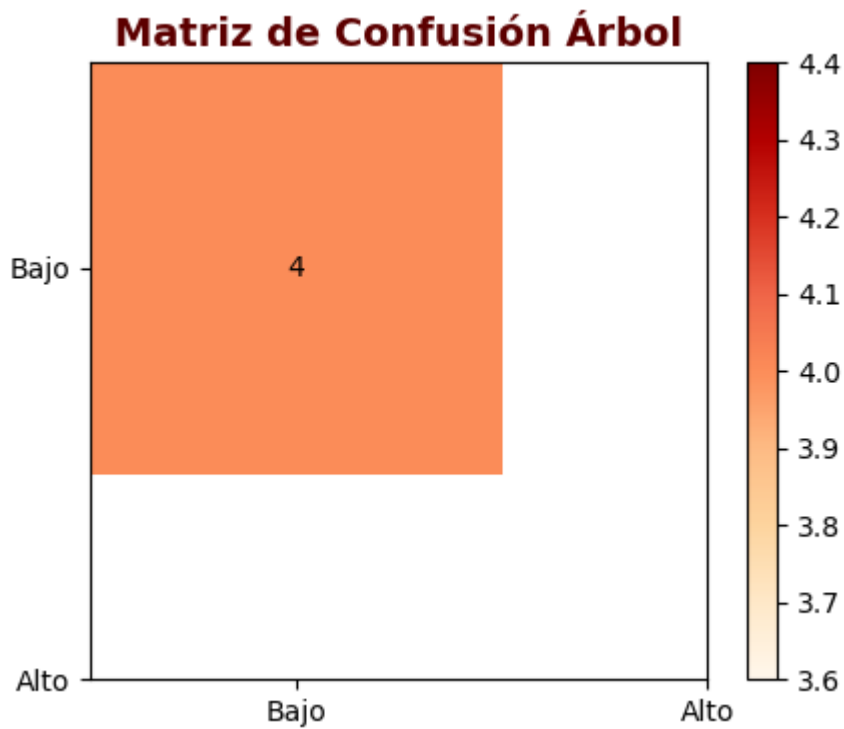
warnings.warn(

```

-----
IndexError                                Traceback (most recent call last)
Cell In[43], line 38
    32 for i in range(2):
    33     for j in range(2):
    35         plt.text(
    36             j,
    37             i,
---> 38             cm_tree[i,j],
    39             ha="center",
    40             va="center",
    41             color="black"
    42         )
    44 plt.xlabel("Predicho")
    45 plt.ylabel("Real")

```

IndexError: index 1 is out of bounds for axis 1 with size 1



COMPARO MODELOS

```
In [45]: accuracy_knn = accuracy_score(
    y_test,
    y_pred_knn
)

accuracy_tree = accuracy_score(
    y_test,
    y_pred_tree
)

comparacion = pd.DataFrame({
    "Modelo": ["KNN", "Árbol"],
    "Accuracy": [
        accuracy_knn,
        accuracy_tree
    ]
})

comparacion
```

Out[45]:

	Modelo	Accuracy
0	KNN	1.0
1	Árbol	1.0

```
In [46]: plt.figure(figsize=(6,4))

plt.bar(
    comparacion["Modelo"],
    comparacion["Accuracy"],
    color=["firebrick", "indianred"]
)
```

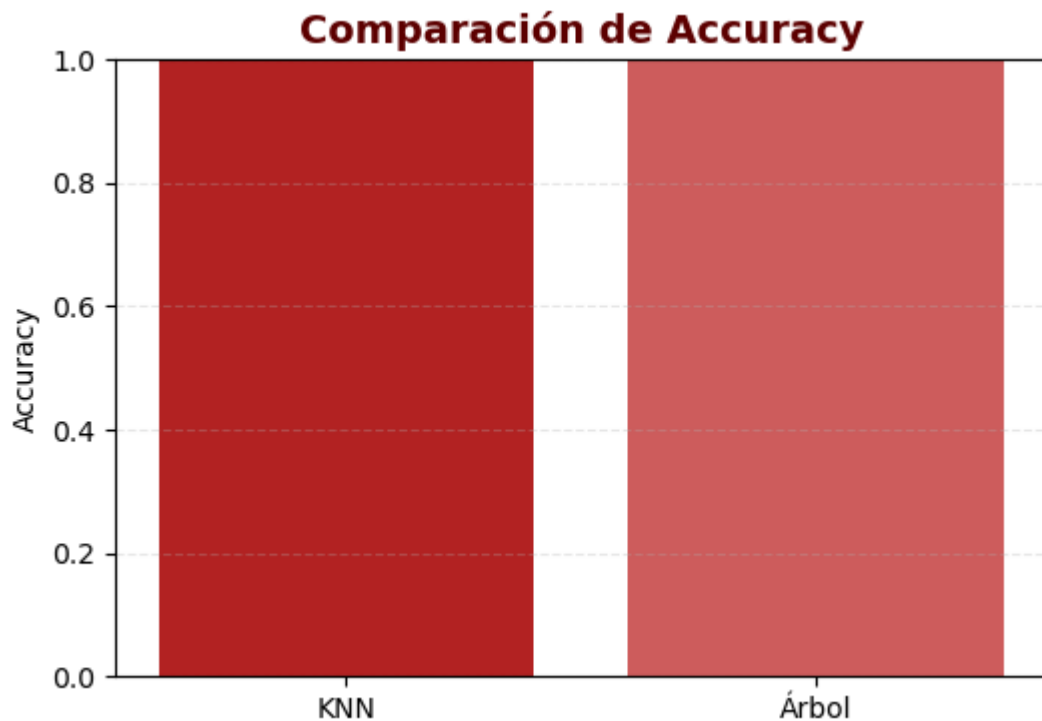
```
plt.title(
    "Comparación de Accuracy",
    fontsize=14,
    color="#5C0000",
    fontweight="bold"
)

plt.ylabel("Accuracy")

plt.ylim(0,1)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.show()
```



Análisis de resultados

KNN

Ventajas:

Considera la cercanía entre registros. Buen desempeño con pocos datos. Sensible a patrones similares

Desventajas:

Depende del escalado. Puede verse afectado por ruido. Árbol de Decisión

Ventajas:

Fácil interpretación

Permite visualizar reglas de decisión

No necesita escalado

Desventajas:

Puede sobreajustarse

Sensible a cambios pequeños en los datos

Conclusiones

Ambos modelos permitieron clasificar correctamente el nivel de consumo de psicofármacos.

El modelo KNN mostró un buen desempeño gracias al escalado y la similitud entre registros.

Por otro lado, el Árbol de Decisión permitió interpretar mejor cómo influyen las variables relacionadas con el consumo de psicofármacos.

Debido a que el dataset es pequeño, los resultados pueden variar fácilmente y existe riesgo de sobreajuste.

Como mejoras futuras podrían aplicarse:

Más registros

Validación cruzada

Ajuste de hiperparámetros

Técnicas de balanceo

Nuevas variables predictoras